

Software Requirements Specification

for

MiniOS Design

Version 4.0

Prepared by Group - 2

Group 2 Members :
Aditya Sharma (B24CS1119)
Mehta Taksh Ashish (B24CS1115)
Ved Mitra Verma (B24CS1086)
Mayank Tiwari (B24CS1098)

March 2026

Contents

1	Introduction	4
1.1	Purpose	4
1.2	Document Conventions	4
1.3	Intended Audience and Reading Suggestions	4
1.4	Product Scope	4
1.5	References	5
1.6	Alpha Managers	5
1.7	MiniOS — Sprint Plan	5
1.7.1	Team	5
1.7.2	Feature to SRS Requirements Mapping	5
1.7.3	Sprint 1 — Foundation & Environment Setup	5
1.7.4	Sprint 2 — Core Logic Implementation	7
1.7.5	Sprint 3 — Integration & Synchronization	7
1.7.6	Sprint 4 — Testing, Polish & Demo	7
1.7.7	Risk Register	8
1.7.8	Summary Timeline & Mapping	8
1.8	Project Management Tools	8
2	Overall Description	9
2.1	Product Perspective	9
2.2	Product Functions	9
2.3	User Classes and Characteristics	9
2.4	Operating Environment	9
2.5	Design and Implementation Constraints	9
2.6	User Documentation	10
2.7	Assumptions and Dependencies	10
3	External Interface Requirements	10
3.1	User Interfaces	10
3.2	Hardware Interfaces	10
3.3	Software Interfaces	10
4	System Features	10
4.1	Dynamic Priority Scheduling	10
4.1.1	Description and Priority	10
4.1.2	Functional Requirements	11
4.2	Paging with Copy-On-Write	11
4.2.1	Description and Priority	11
4.2.2	Stimulus/Response Sequences	11
4.2.3	Functional Requirements	12
4.3	File Management System	12
4.3.1	Description	12
4.3.2	Functional Requirements	12
4.4	Garbage Collection	12
4.4.1	Description	12
4.4.2	Functional Requirements	12
4.5	Dynamic Memory Compaction	13

4.5.1	Description	13
4.5.2	Functional Requirements	13
5	Other Nonfunctional Requirements	13
5.1	Performance	13
5.2	Safety Requirements	13
5.3	Security Requirements	13
5.4	Software Quality Attributes	13
A	Glossary	14
B	Analysis Models	14
C	To Be Determined List	14
D	Appendices	15
	Appendices	15

Revision History

Name	Date	Reason for Changes	Version
Group 2	Jan 21 2026	Initial SRS Creation	1.0
Group 2	Feb 4 2026	Clarified deliverables and system goals	2.0
Group 2	March 2 2026	Scheduler, COW Memory design	3.0
Group 2	March 12 2026	IPC removed. Added File Manager, Garbage Collector and Memory Compaction	4.0

1 Introduction

1.1 Purpose

This document specifies the Software Requirements Specification (SRS) for MiniOS Version 4.0.

MiniOS is a pedagogical operating system built by extending the xv6 UNIX-like teaching OS. The system demonstrates scheduling, memory management, file management, garbage collection, and dynamic memory compaction mechanisms.

1.2 Document Conventions

Requirements are labeled:

REQ-SCH-* (Scheduling)

REQ-MEM-* (Memory)

REQ-FILE-* (File Manager)

REQ-GC-* (Garbage Collector)

REQ-DMEM-* (Dynamic Memory Compaction)

NFR-PERF-* (Nonfunctional Performance)

NFR-SAFE-* (Nonfunctional Safety)

NFR-SEC-* (Nonfunctional Security)

NFR-QUAL-* (Nonfunctional Software Quality)

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- OS Developers
- Academic Evaluators
- System Testers
- Project Supervisors

Readers should begin with [Section 2 \(Overall Description\)](#) before proceeding to System Features.

1.4 Product Scope

MiniOS extends xv6 to demonstrate:

- Priority-based scheduling with aging
- Paging with Copy-On-Write
- File management subsystem
- Garbage collection
- Dynamic memory compaction

1.5 References

1. Arpaci-Dusseau, Operating Systems: Three Easy Pieces.
2. IIT Bombay OS Notes.
3. MIT xv6 Documentation.
4. OSDev Wiki.

1.6 Alpha Managers

- Mehta Taksh Ashish – Software Systems, Team
- Ved Mitra Verma – Work , Way of Working
- Mayank Tiwari – Stakeholder, Requirements
- Aditya Sharma – Opportunity

1.7 MiniOS — Sprint Plan

Project: MiniOS (xv6 Extension)

Platform: xv6 kernel running on QEMU x86 emulator

1.7.1 Team

#	Member	Feature Ownership
1	Taksh Mehta	Dynamic Priority Scheduler
2	Aditya Sharma	Paging + Copy-On-Write (Lead)
3	Mayank Tiwari	Dynamic Memory + Compaction (Integration)
4	Ved Mitra Verma	File Manager + Garbage Collector

1.7.2 Feature to SRS Requirements Mapping

Feature	SRS Requirements
Dynamic Priority Scheduling	REQ-SCH-1, REQ-SCH-2, REQ-SCH-3, REQ-SCH-4, REQ-SCH-5, REQ-SCH-6, REQ-SCH-7
Paging with Copy-On-Write	REQ-MEM-1, REQ-MEM-2, REQ-MEM-3, REQ-MEM-4, REQ-MEM-5, REQ-MEM-6
File Manager	REQ-FILE-1, REQ-FILE-2, REQ-FILE-3, REQ-FILE-4
Garbage Collector	REQ-GC-1, REQ-GC-2, REQ-GC-3
Dynamic Memory Compaction	REQ-DMEM-1, REQ-DMEM-2, REQ-DMEM-3

1.7.3 Sprint 1 — Foundation & Environment Setup

Duration: Week 1

Goal: Working development environment and subsystem skeletons integrated into xv6.

Tasks: Taksh — Priority Scheduler

- Study `proc.c`, `trap.c`, `sched()` workflow
- Extend `struct proc` with `priority`, `wait_ticks`, `cpu_ticks`
- Implement scheduler skeleton
- Define initial α and β parameters

Aditya — COW Memory (Lead)

- Study xv6 paging implementation in `vm.c`
- Study `fork()` memory behavior
- Add reference counting array `ref_count[PHYSTOP/PGSIZE]`
- Draft Copy-On-Write page fault design

Mayank — Dynamic Memory Management

- Implement memory allocation structures
- Study `kalloc.c`
- Add dynamic memory tracking structures
- Design memory compaction algorithm

Ved Mitra — File Manager + Garbage Collector

- Design basic file abstraction structure
- Implement file creation and deletion stubs
- Design garbage collector mechanism
- Add system call stubs for file operations

Sprint 1 Deliverables

- xv6 builds and runs on all machines
- `struct proc` extended successfully
- Reference count structure implemented
- File manager structure defined
- Initial memory management structures implemented
- Kernel boots without regression

1.7.4 Sprint 2 — Core Logic Implementation

Duration: Week 2

Goal: Implement core functionality for scheduler, file management, and memory handling.

Sprint 2 Deliverables

- Scheduler selects highest priority process
- Fork shares memory pages using COW
- File creation, read, write, and deletion operations implemented
- Garbage collector detects unused memory blocks
- Arithmetic and logical processes successfully executed

1.7.5 Sprint 3 — Integration & Synchronization

Duration: Week 3,4

Goal: Integrate all subsystems and resolve edge cases.

Cross-Feature Coordination Points

Touchpoint	Who	Risk	Resolution
<code>struct proc</code> layout	Taksh + Aditya	Field conflicts	Freeze layout by Sprint 3
Memory allocation integration	Mayank + Aditya	Memory inconsistency	Shared allocation interface
File storage and memory blocks	Ved + Mayank	Fragmentation issues	Memory compaction logic
Scheduler interaction with GC	Taksh + Ved	Timing conflicts	Run GC during idle cycles

Sprint 3 Deliverables

- No starvation under scheduler stress tests
- COW memory stable under concurrent writes
- File manager integrated with dynamic memory system
- Garbage collector safely frees unused memory
- Memory compaction reduces fragmentation
- No kernel panic under combined workloads

1.7.6 Sprint 4 — Testing, Polish & Demo

Duration: Week 5

Goal: Full validation, documentation, and system demonstration.

Sprint 4 Deliverables

- Scheduler performance validated
- Copy-On-Write memory stress tests pass
- File operations validated with multiple processes
- Garbage collection successfully reclaims memory
- Memory compaction verified during idle state
- Arithmetic/Logical workload runs correctly
- QEMU demo prepared

1.7.7 Risk Register

Risk	Likelihood	Impact	Mitigation
α/β instability	Medium	Medium	Parameter tuning during testing
COW race on <code>ref_count</code>	High	High	Spinlocks for synchronization
Memory fragmentation	Medium	Medium	Dynamic compaction algorithm
Garbage collection timing issues	Medium	Medium	Run GC during idle CPU cycles
QEMU memory limit	Low	Medium	Define memory constraints early

1.7.8 Summary Timeline & Mapping

Week	Semester Timeline	Sprint	Focus
Week 1	March 2, 2026 - March 8, 2026	Sprint 1	Setup + Subsystem Stubs
Week 2	March 9, 2026 - March 15, 2026	Sprint 2	Core Logic Implementation
Week 3,4	March 16, 2026 - March 29, 2026	Sprint 3	Integration + Synchronization
Week 5	March 30, 2026 - April 5, 2026	Sprint 4	Testing + Demo

1.8 Project Management Tools

The MiniOS project will be developed using modern project management and collaboration tools to ensure structured planning, effective coordination, and clear documentation.

- **Trello** will be used for task planning and progress tracking through boards and cards representing different development stages such as design, implementation, and testing.
- **Notion** will be used as a centralized platform for maintaining design documentation, architectural decisions, and development notes.
- **Git** will be used for version control to manage source code changes, track revisions, and support collaborative development.
- **QEMU** and the GNU toolchain will be used for building, testing, and debugging the operating system in an emulated environment.

2 Overall Description

2.1 Product Perspective

MiniOS is built as an extension of xv6 and runs inside the QEMU x86 emulator. It modifies scheduling, memory management, and IPC subsystems while retaining xv6's core kernel structure.

2.2 Product Functions

MiniOS provides:

- Dynamic priority scheduling
- Paging with Copy-On-Write
- File storage and retrieval
- Memory reclamation using garbage collection
- Dynamic memory compaction

2.3 User Classes and Characteristics

- Students learning OS concepts
- Developers modifying xv6
- Evaluators analyzing subsystem design

2.4 Operating Environment

- QEMU x86 emulator
- xv6 kernel base
- GNU toolchain
- C and x86 Assembly

2.5 Design and Implementation Constraints

- Must extend xv6 architecture
- Must run under QEMU constraints
- Limited simulated memory
- Paging structure inherited from xv6

2.6 User Documentation

- Design documentation
- UML diagrams

2.7 Assumptions and Dependencies

- xv6 paging structure is available
- QEMU provides stable hardware simulation
- Fork-heavy workloads assumed for COW benefit

3 External Interface Requirements

3.1 User Interfaces

MiniOS interacts via system calls and terminal interface inherited from xv6.

3.2 Hardware Interfaces

Runs on simulated x86 hardware via QEMU.

3.3 Software Interfaces

- xv6 kernel
- Page tables
- Interrupt handling subsystem

4 System Features

4.1 Dynamic Priority Scheduling

4.1.1 Description and Priority

Implements preemptive scheduling with aging to prevent starvation. Priority: High

- Priority update rule:
 - $p_i(t)$ = dynamic priority
 - α = aging increment
 - β = CPU penalty

$$p_i(t+1) = \begin{cases} p_i(t) - \beta & \text{if CPU-bound} \\ p_i(t) + \alpha & \text{if waiting} \end{cases}$$

4.1.2 Functional Requirements

- REQ-SCH-1: The system shall assign a dynamic priority $p_i(t)$ to each process.
- REQ-SCH-2: Waiting processes shall gain priority using aging parameter α .
- REQ-SCH-3: CPU-bound processes shall be penalized using parameter β .
- REQ-SCH-4: The system shall guarantee bounded waiting time.
- REQ-SCH-5: The system shall define excessive CPU usage as CPU execution exceeding T_{CPU_MAX} timer ticks within a scheduling window W .
 - Timer Tick: A periodic hardware interrupt generated by the system clock used as the base unit of time measurement.
 - Scheduling Window (W): A logical grouping of timer ticks used to evaluate process CPU usage behavior for priority adjustment.
- REQ-SCH-6: The default scheduling parameter values shall be:
 - $\alpha_{default} = 1$
 - $\beta_{default} = 2$
 - $T_{CPU_MAX_default} = 10$ timer ticks
 - $W_{default} = 50$ timer ticks
- REQ-SCH-7: The allowed configuration ranges shall be:
 - $0 \leq \alpha \leq 5$
 - $0 \leq \beta \leq 10$
 - $1 \leq T_{CPU_MAX} \leq 100$
 - $10 \leq W \leq 500$

4.2 Paging with Copy-On-Write

4.2.1 Description and Priority

Implements memory sharing after fork and duplication upon write. Priority: High

4.2.2 Stimulus/Response Sequences

- fork() called $\rightarrow$ pages shared
- write to shared page $\rightarrow$ page fault $\rightarrow$ copy created

4.2.3 Functional Requirements

- REQ-MEM-1: The system shall share pages after fork.
- REQ-MEM-2: The system shall copy a page only upon modification.
- REQ-MEM-3: The system shall maintain reference counting.
- REQ-MEM-4: The system shall handle page faults correctly.
- REQ-MEM-5: The system shall use a fixed page size of 4096 bytes (4KB), consistent with x86 paging architecture in xv6.
- REQ-MEM-6: Page size shall remain fixed and shall not be configurable in Version 1.0.
- Memory Model:
 - N = pages in parent process
 - S = page size
 - k = modified pages

$$Memory_{COW} = (N + k)S$$

4.3 File Management System

4.3.1 Description

Responsible for file storage, access, and deletion.

4.3.2 Functional Requirements

- REQ-FILE-1: System shall allow file creation.
- REQ-FILE-2: System shall support reading and writing files.
- REQ-FILE-3: System shall allow file deletion.
- REQ-FILE-4: Files shall be stored in dynamically allocated memory.

4.4 Garbage Collection

4.4.1 Description

Garbage collector reclaims memory from deleted files.

4.4.2 Functional Requirements

- REQ-GC-1: System shall detect unused file blocks.
- REQ-GC-2: Garbage collector shall free unused memory.
- REQ-GC-3: Garbage collection shall run during system idle periods.

4.5 Dynamic Memory Compaction

4.5.1 Description

Compacts memory to remove fragmentation.

4.5.2 Functional Requirements

- REQ-DMEM-1: When files are deleted the system shall compact memory.
- REQ-DMEM-2: When the system is idle full memory compaction shall occur.
- REQ-DMEM-3: File references shall be updated after relocation.

5 Other Nonfunctional Requirements

5.1 Performance

- NFR-PERF-1: The scheduler shall guarantee bounded waiting time such that no runnable process waits more than W_{MAX} timer ticks under normal system load.
- NFR-PERF-2: The Copy-On-Write mechanism shall reduce physical memory duplication by at least 30% compared to naive fork implementation under fork-heavy workloads.
- NFR-PERF-3: Memory compaction shall reduce fragmentation.

5.2 Safety Requirements

- NFR-SAFE-1: No memory corruption shall occur due to concurrent page fault handling in Copy-On-Write operations.
- NFR-SAFE-2: Invalid memory access shall return an error code and shall not cause kernel panic.

5.3 Security Requirements

- NFR-SEC-1: No process shall access another process's memory without explicit kernel mediation.

5.4 Software Quality Attributes

- NFR-QUAL-1 (Modularity): The system shall maintain architectural separation between the Scheduler, Memory Manager, and IPC subsystems such that modifications to one subsystem do not require changes in unrelated subsystems.
- NFR-QUAL-2 (Correctness): All implemented features shall conform to their respective functional requirements (REQ-SCH-*, REQ-MEM-*, REQ-IPC-*), and no kernel panic shall occur during normal operation under specified workloads.

- NFR-QUAL-3 (Extensibility): The system shall allow addition of new scheduling policies or IPC mechanisms without requiring redesign of the core process abstraction.
- NFR-QUAL-4 (Reliability): The system shall operate without memory corruption, deadlock, or kernel crash during continuous execution for at least 10,000 timer ticks under combined subsystem workload.

A Glossary

- Paging: Fixed size memory management technique.
- Page Size (S): Fixed memory unit of 4096 bytes used for virtual memory paging.
- Copy-On-Write: Delayed memory copying.
- Garbage Collection: Automatic memory reclamation.
- Memory Compaction: Reorganizing memory to reduce fragmentation.
- Aging: Priority improvement over waiting time.
- Starvation: Indefinite waiting.

B Analysis Models

UML diagrams include:

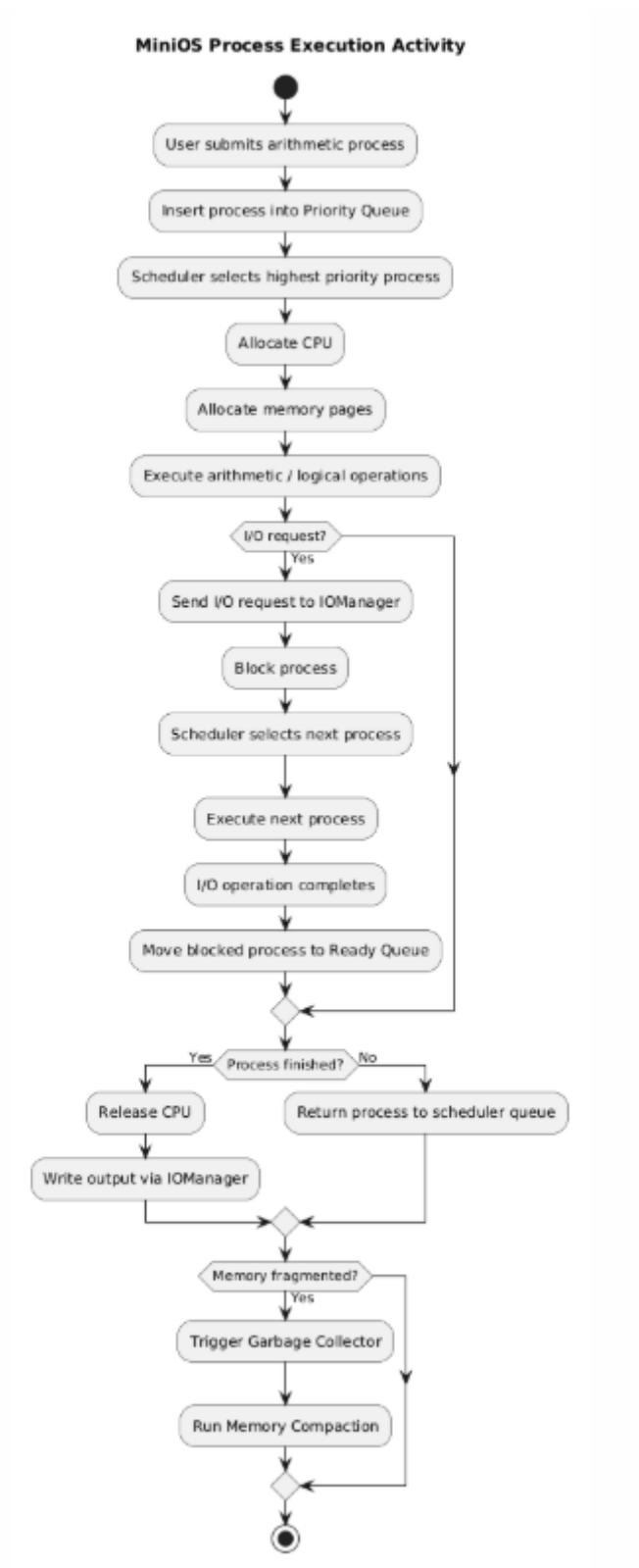
- [Class Diagram](#)
- [Component Diagram](#)
- [Composite Diagram](#)
- [Activity Diagram](#)
- [Sequence](#)
- [Timing](#)
- [Deployment Diagram](#)
- [Use Case Diagram](#)

C To Be Determined List

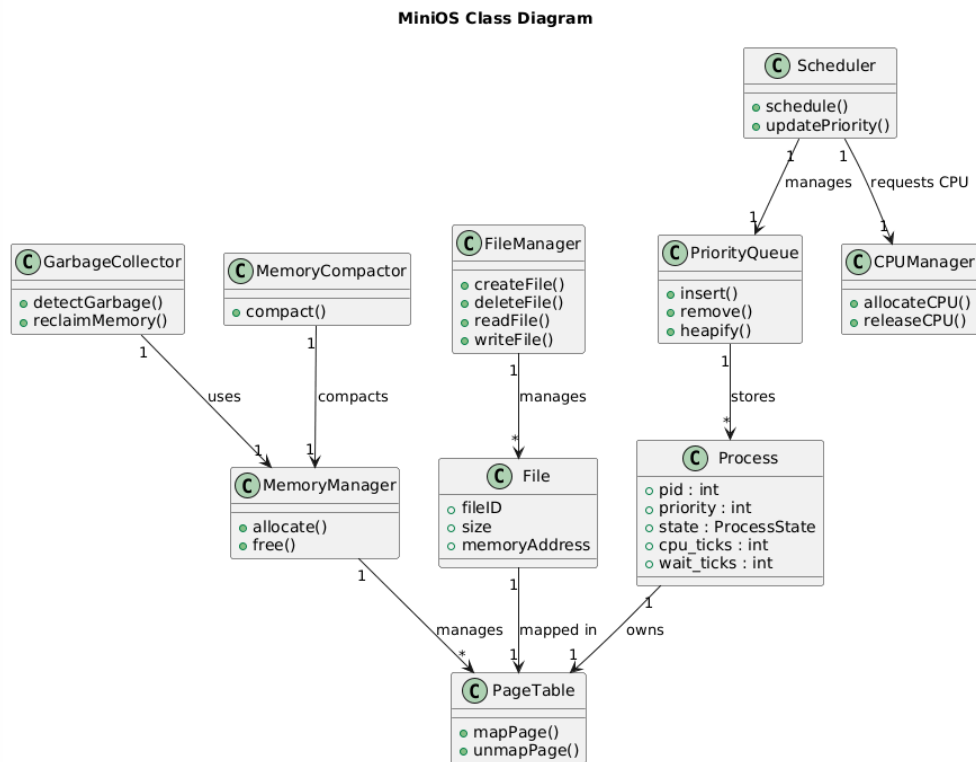
- Synchronization primitive specifications
- Memory limit configuration under QEMU

D Appendices

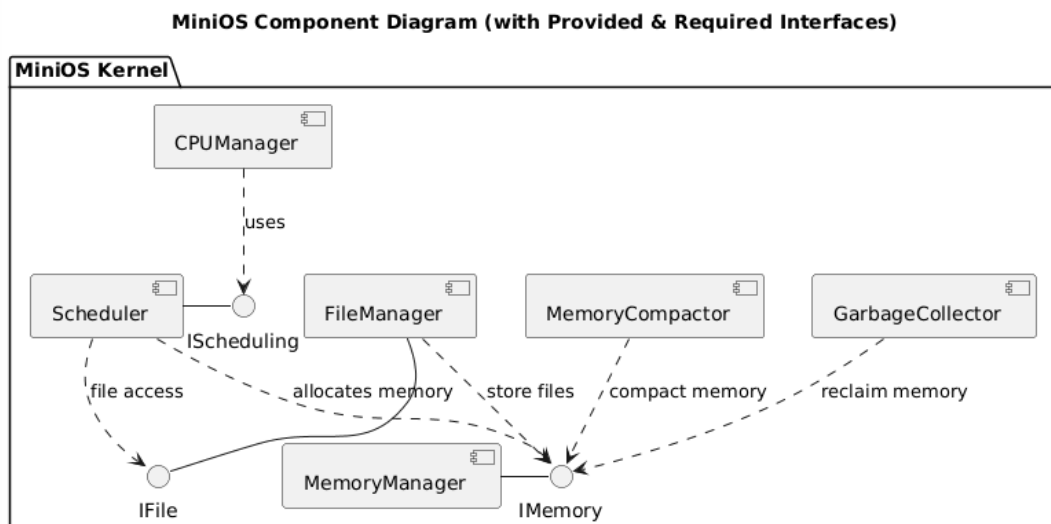
Activity Diagram



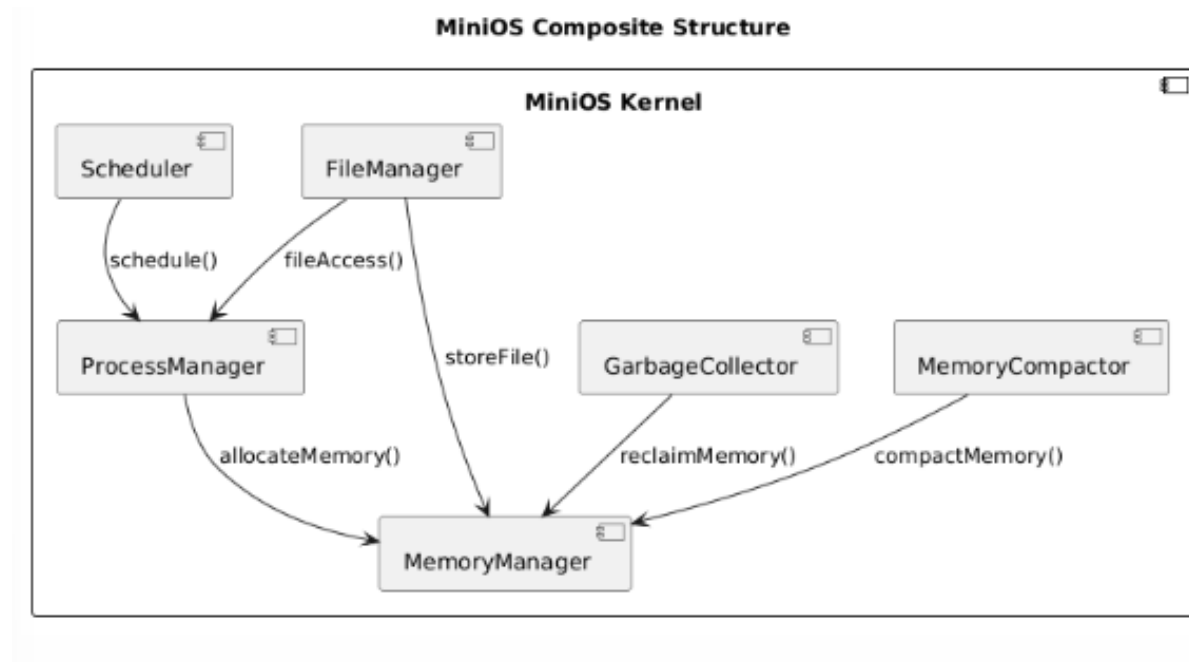
Class Diagram



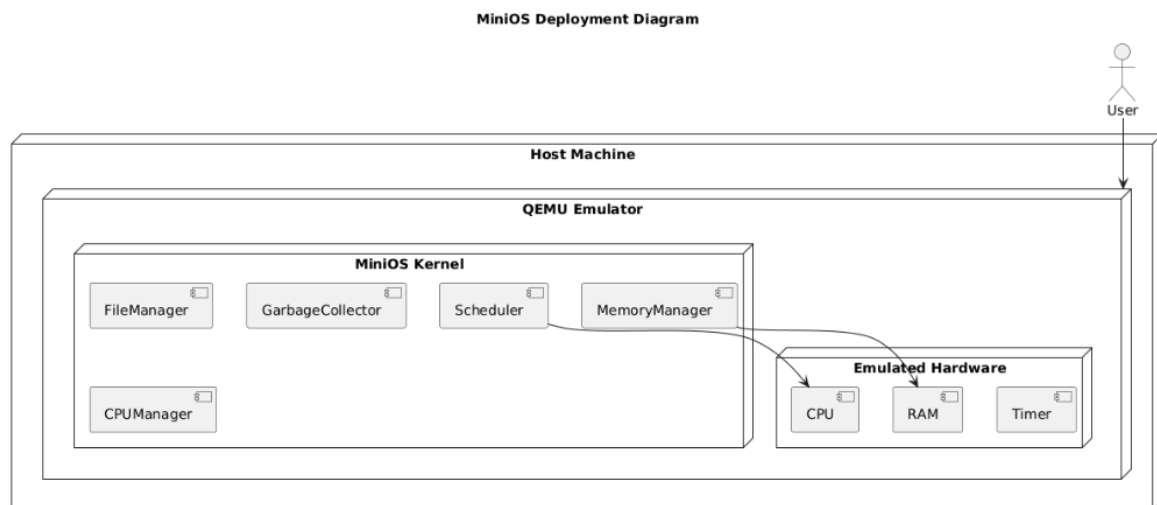
Component Diagram



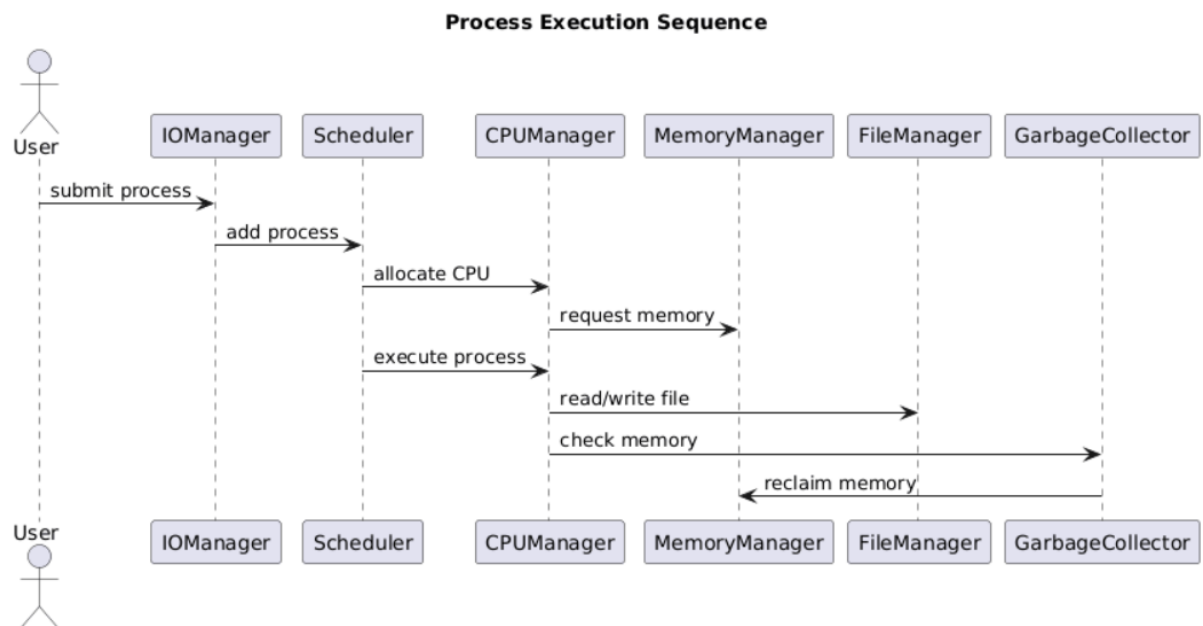
Composite Diagram



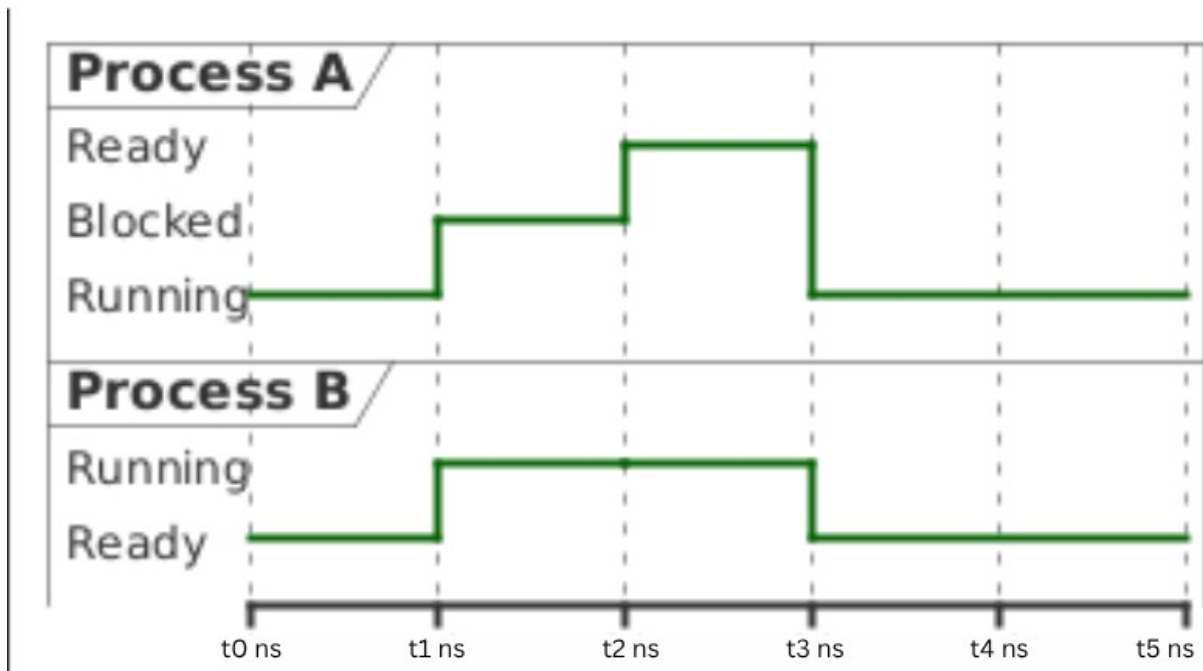
Deployment Diagram



Sequence Diagram



Timing Diagram



Use Case Diagram

